

Génération de plans architecturaux 2D par cGAN piloté par prompt et entraîné en Split Learning : vers une CAO générative collaborative et confidentielle

Kenny Tshibangu Ntumba^{1,*}

¹ Université Nouveaux Horizons, Faculté des Sciences Informatiques, Ingénierie Logicielle, Lubumbashi, République Démocratique du Congo

Résumé

Contexte : les logiciels de conception assistée par ordinateur (CAO) aident à dessiner des plans, mais ne savent pas encore proposer d'esquisse à partir d'une demande formulée en langage courant. Par ailleurs, entraîner un modèle génératif suppose en général de rassembler des plans qui, pour les agences d'architecture, constituent des données propriétaires. **Objectif :** concevoir et évaluer un générateur de plans résidentiels 2D piloté par une requête textuelle, qui puisse être entraîné par plusieurs agences sans qu'aucune ne transmette ses données. **Méthodes :** la requête est convertie en contraintes partielles portant sur le programme (nombre de pièces de chaque type) et sur la structure du graphe de bulles (suite parentale, cuisine fermée, etc.). Un GAN conditionnel (cGAN) complète les éléments laissés libres, pièces comme topologie, puis les contraintes explicites sont rétablies et un placement procédural guidé par le graphe construit le plan. Le cGAN est entraîné de façon centralisée, puis en Split Learning en U entre trois agences aux données hétérogènes et un serveur, avec ou sans bruit sur les activations échangées. **Résultats :** sur 20 000 programmes, la méthode respecte 100,0 % des contraintes de la requête dans le plan final, avec 80,5 % de programmes plausibles, 99,1 % de portes conformes et un temps médian de 2,1 ms par plan. La

prédiction du graphe de bulles fait passer le respect des structures demandées de 50,8 % à 99,4 %. En SplitFed, le générateur atteint 97,9 % de respect des contraintes, contre 100,0 % en centralisé et 97,0 % pour une agence travaillant seule. Cependant, une attaque par inversion retrouve 52,9 % des programmes à partir des activations échangées ; l'ajout d'un bruit gaussien ramène ce taux à 0,4 % au prix d'environ deux points de respect. Conclusion : le Split Learning permet à plusieurs agences d'obtenir ensemble un générateur proche du modèle centralisé et meilleur que leurs modèles locaux, à condition de protéger explicitement les activations échangées.

Keywords: GAN conditionnel, Split Learning, SplitFed, génération de plans architecturaux, conception assistée par ordinateur, confidentialité des données, requête textuelle

1 Introduction

La conception assistée par ordinateur (CAO) occupe aujourd'hui une place centrale en architecture et en ingénierie [1]. Les logiciels les plus répandus, comme AutoCAD, Revit ou ArchiCAD, restent toutefois avant tout des outils de dessin. L'architecte y trace chaque mur et explore les variantes les unes après les autres, ce qui rend la phase d'esquisse longue et coûteuse. Dans ce contexte, les réseaux antagonistes génératifs (GANs) [2] ont montré qu'il était possible d'apprendre la distribution d'un ensemble de plans et d'en proposer de nouveaux [3–6].

Deux difficultés freinent cependant leur usage en agence. La première tient au contrôle de la génération : un architecte souhaite décrire ce qu'il veut, par exemple « un T3 avec balcon », plutôt que tirer des plans au hasard. La seconde tient à la nature des données. Les plans d'une agence font partie de son

savoir-faire, et elle n'acceptera pas facilement de les confier à un tiers pour entraîner un modèle commun.

Le *Split Learning* (SL) [7, 8] apporte une réponse à ce second problème. Le réseau y est partagé entre le client, qui conserve ses données, et un serveur, de sorte que seules les activations intermédiaires (*smashed data*) et leurs gradients transitent entre les deux. À notre connaissance, cette approche n'a pas encore été appliquée à la génération de plans architecturaux.

Ce travail prolonge le prototype développé dans notre mémoire de Master (2025). Celui-ci associait un générateur encodeur-décodeur à un discriminateur convolutif opérant sur des images de plans, avec des pertes d'adjacence et de similarité structurelle (SSIM). Il comportait toutefois une faiblesse importante : le vecteur produit par le générateur passait par une fonction de placement non différentiable avant d'être évalué, si bien que ces pertes ne pouvaient pas réellement guider l'apprentissage. Le Split Learning n'y était par ailleurs qu'esquissé. Nous en proposons ici une refonte complète, dont les apports sont les suivants :

1. une chaîne *requête* $\rightarrow$ *cGAN* $\rightarrow$ *placement* dans laquelle le réseau apprend le programme (quelles pièces et combien), tandis qu'un placement guidé par un graphe de bulles assure la validité du plan : pièces accessibles, une porte par pièce privée, absence de liaisons interdites et respect de la surface demandée ;
2. un cGAN qui prédit à la fois le programme et la topologie du graphe de bulles, décrite par six variables de rattachement, et dont la condition distingue les éléments imposés des éléments libres, ce qui permet de piloter aussi la structure du logement par la requête ;
3. une mise en œuvre complète d'un Split Learning en U entre plusieurs agences et un serveur, sans que les programmes ni les requêtes ne quittent les agences, comparée à l'entraînement centralisé, à l'entraînement local et à SplitFed [9] ;
4. un protocole d'évaluation reproductible, comprenant des méthodes de référence, des ablations, une mesure du coût de communication et une attaque par inversion, dont tous les chiffres sont produits par un notebook public.

2 Travaux connexes

2.1 Génération de plans par apprentissage profond

Huang et Zheng [3], puis Chaillou [5], ont abordé la génération de plans comme un problème de traduction d'images (pix2pix), en passant successivement de l'emprise au zonage puis à l'ameublement. Zheng et al. [4] ont appliqué cette démarche à des appartements chinois et japonais et ont souligné sa sensibilité au style des données d'entraînement. Les travaux plus récents conditionnent la génération par la structure du programme. House-GAN [6] prend ainsi en entrée un graphe de bulles décrivant les pièces et leurs adjacences, et Graph2Plan [10] combine ce graphe avec l'emprise du bâtiment en s'appuyant sur le jeu de données RPLAN [11]. Ces approches supposent néanmoins que l'utilisateur fournisse un graphe complet, alors que nous partons d'une requête partielle que le modèle doit compléter. Enfin, Bank et al. [12] et Kim et al. [13] ont montré l'intérêt des GANs pour l'exploration créative, sans imposer de contrainte fonctionnelle explicite.

2.2 Split Learning et confidentialité

En Split Learning [7, 8], le client calcule les premières couches du réseau et transmet les activations obtenues au serveur. Dans la variante dite « en U », il conserve également les dernières couches ainsi que les étiquettes. Comparé à l'apprentissage fédéré (FedAvg [14]), le Split Learning réduit le calcul effectué par le client et peut diminuer le volume échangé lorsque le modèle est grand par rapport aux activations [15]. SplitFed [9] combine les deux approches en moyennant les parties client. Les activations échangées ne sont cependant pas sans risque, puisque des attaques permettent d'en reconstruire les données ou de détourner l'entraînement [16–18], ce qui a conduit à proposer des défenses par décorrélation ou par bruitage [19]. Une revue récente [20] recense les domaines d'application du Split Learning, notamment la santé et l'Internet des objets [21], mais n'en mentionne aucun en CAO.

2.3 Positionnement

Notre objectif n'est pas de rivaliser avec House-GAN ou Graph2Plan sur la qualité géométrique des plans. Nous cherchons plutôt à savoir s'il est possible d'apprendre, sans centraliser les données d'agences concurrentes, un modèle capable de transformer une demande textuelle incomplète en un programme réaliste, puis en un plan valide.

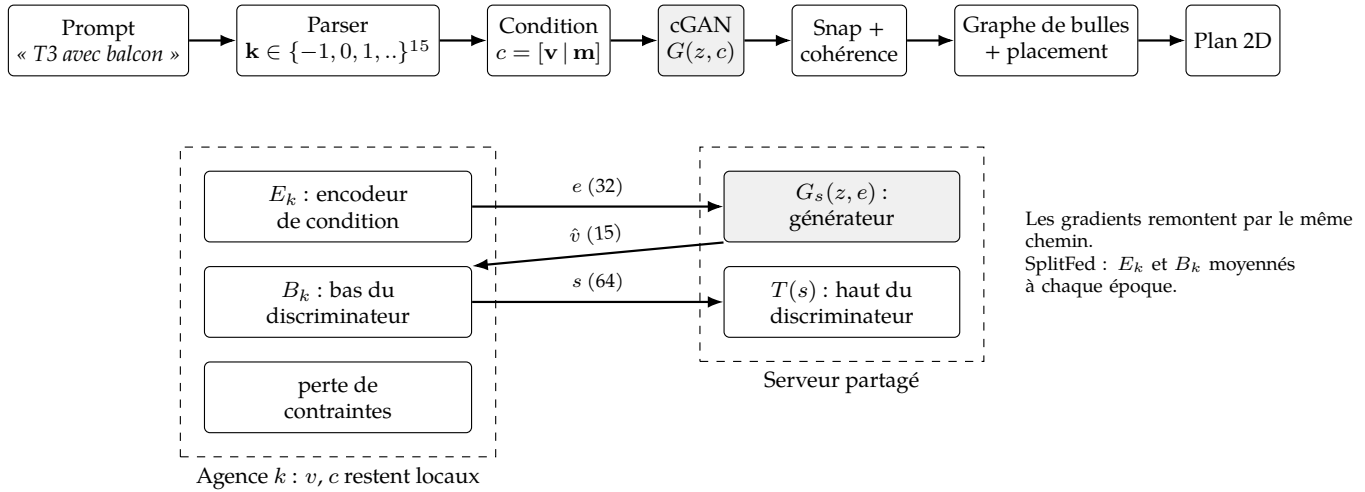


Figure 1. En haut, le pipeline d’inférence. En bas, le découpage du cGAN pour le Split Learning en U : seuls les vecteurs e , s , le vecteur généré $\hat{v}$ et leurs gradients franchissent la frontière ; les programmes réels v et les conditions c restent chez l’agence.

3 Méthode

3.1 Représentation du programme et du graphe de bulles

Nous appelons *programme* un vecteur $v \in \mathbb{N}^{15}$. Ses neuf premières composantes donnent le nombre de pièces de chaque type (salon, chambre, douche, toilettes, couloir, bureau, cuisine, balcon, dressing), et les six suivantes décrivent la topologie du graphe de bulles, c’est-à-dire l’arbre de distribution du logement. Plutôt que de générer une matrice d’adjacence de taille variable comme le fait House-GAN [6], nous sommes partis du constat que, dans un logement, la liberté de cet arbre porte essentiellement sur le rattachement des pièces annexes. Six entiers suffisent donc à le décrire : le nombre de salles d’eau privatives (suites), de WC intégrés à la salle d’eau, de dressings attenants à une chambre, de balcons desservis par une chambre, de cuisines desservies par le couloir (cuisine fermée) et de bureaux ouverts sur le séjour. Cette représentation reste compacte, se traduit toujours en un arbre valide une fois ramenée dans le domaine réalisable (par exemple, pas plus de suites que de chambres) et se prête directement à un perceptron multicouche.

Chaque type de pièce dispose d’une plage de dimensions sur une grille dont la cellule mesure 30 cm (de 16 à 26 m² pour le séjour, de 8 à 13 m² pour une chambre) et d’une liste de voisins autorisés. Une chambre peut ainsi s’ouvrir sur un couloir, une salle d’eau, un dressing ou un balcon, et dans les petits logements le séjour joue le rôle de dégagement. Nous avons également défini des liaisons interdites : WC ou salle d’eau sur la cuisine, chambre sur chambre ou sur

cuisine, WC sur une chambre ou un bureau, et WC sur le séjour dès lors qu’un couloir existe. Le générateur travaille uniquement sur v , la géométrie étant confiée au placement (§3.5). Ce choix découle directement de l’expérience du prototype v1 : il rend la perte de contraintes différentiable et confie les garanties fonctionnelles à un algorithme que l’on peut vérifier, plutôt qu’au réseau.

3.2 De la requête à la condition

Le parser associe à chaque type de pièce une liste de synonymes (« séjour », « sdb », « wc », « terrasse », etc.). Il reconnaît les typologies (studio, T1 à T6), les nombres écrits en chiffres ou en lettres, les négations comme « sans balcon », ainsi qu’une surface ou une emprise (« T3 de 65 m² », « plan 9×11 m »). Certaines expressions fixent en outre la topologie : « suite parentale », « cuisine fermée » ou « ouverte », « WC séparés », « WC dans la salle de bain », « chambre avec dressing » ou « coin bureau ». Lorsqu’une telle expression suppose des pièces non mentionnées, celles-ci sont ajoutées ; une suite parentale implique par exemple deux salles d’eau. Le parser produit ainsi un vecteur $k \in \{-1, 0, 1, \dots\}^{15}$, où -1 indique une valeur libre. La condition transmise au cGAN sépare la valeur du masque :

$$c = [v \mid m], \quad m_i = \mathbf{1}[k_i \geq 0], \quad v_i = m_i \min(1, k_i/M_i), \quad (1)$$

où M_i désigne le maximum observé pour la composante i dans les données. Sans ce masque, le réseau ne pourrait pas distinguer une demande « sans balcon » d’une requête qui ne mentionne pas le balcon.

3.3 GAN conditionnel

Le générateur $G(z, c)$ et le discriminateur $D(v, c)$ sont des perceptrons multicouches comptant respectivement 34319 et 30721 paramètres, avec $z \in \mathbb{R}^{32}$. Pendant l'entraînement, chaque programme réel est associé à une condition obtenue en masquant aléatoirement entre 30 et 90 % de ses composantes, ce qui apprend au modèle à compléter des requêtes plus ou moins détaillées. Le générateur minimise

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}} + \lambda \frac{\sum_i m_i (G(z, c)_i - v_i)^2}{\sum_i m_i}, \quad (2)$$

où le second terme pénalise l'écart aux valeurs imposées. Nous avons retenu $\lambda = 15$, valeur dont l'influence est étudiée plus loin. Le discriminateur est entraîné avec une entropie croisée et un lissage des étiquettes (0,9 et 0,1).

3.4 Rétablissement des contraintes

À l'inférence, la sortie du générateur est dénormalisée et arrondie, puis les valeurs imposées par la requête sont rétablies telles quelles (étape que nous appelons *snap*). Trois règles minimales s'appliquent ensuite : un salon et une cuisine sont toujours présents, et un couloir est ajouté à partir de deux chambres. Le snap assure le respect des demandes explicites, y compris lorsqu'elles sortent de la distribution apprise (cinq chambres, alors que les données n'en comptent pas plus de quatre), tandis que le cGAN reste responsable de tout ce que la requête ne précise pas.

3.5 Placement guidé par le graphe de bulles

La version précédente du placement (v2), héritée du mémoire, accolait chaque pièce à un voisin autorisé choisi au hasard, puis ouvrait une porte sur chaque contact autorisé. Sur 300 programmes, nous avons constaté qu'elle produisait en moyenne 1,9 porte par chambre et 1,6 par salle d'eau, transformant ces pièces en pièces de passage. Elle laissait aussi 17 % des chambres sans accès direct à une circulation, faisait ouvrir 10 % des WC sur le séjour ou la cuisine, et donnait une emprise très découpée (compacité de 0,66). Le placement v3 que nous proposons se déroule en quatre étapes.

Construction du graphe. Le programme et sa topologie permettent d'abord de construire l'arbre de distribution du logement, dont le séjour est la racine. Le couloir, lorsqu'il existe, y est rattaché et dessert les chambres, la salle d'eau commune, les WC et le bureau; en son absence, ce rôle revient au séjour. Les six variables de topologie déterminent ensuite les

autres rattachements : une salle d'eau supplémentaire peut être placée sur la circulation ou en suite, les WC sur le couloir ou dans la salle d'eau commune, le dressing et le balcon sur une chambre ou sur une circulation, la cuisine sur le séjour ou sur le couloir, et le bureau sur le couloir ou sur le séjour. Une salle d'eau commune est toujours conservée. Lorsque aucune topologie n'est fournie, des règles fixes s'appliquent; elles nous servent de référence dans l'ablation.

Placement des pièces. Les pièces sont posées dans l'ordre de l'arbre, chacune contre sa pièce parente. Leur taille est tirée dans leur plage, puis mise à l'échelle lorsque la requête précise une surface, selon le facteur $s = \sqrt{S_{\text{demandée}}/S_{\text{attendue}}}$ borné à $[0,75; 1,5]$. La longueur du couloir dépend du nombre de pièces qu'il dessert. Parmi les positions qui touchent la pièce parente sur au moins 90 cm sans chevauchement, nous retenons la plus proche du centre des pièces déjà placées. Si la pièce parente n'a plus de place, les autres pièces autorisées sont essayées, puis les autres pièces non interdites.

Portes. Les portes forment un arbre couvrant de coût minimal sur le graphe des contacts entre pièces. Le coût d'un contact vaut 0 s'il correspond à une arête du graphe de bulles, 1 s'il s'agit d'un contact autorisé avec une circulation, 2 pour un autre contact autorisé, 10 pour un contact neutre et 100 pour un contact interdit. Seule la zone jour (séjour, cuisine, balcon, couloir) peut recevoir des portes supplémentaires formant une boucle, à moins que cela ne contredise le graphe; une cuisine fermée ne s'ouvre donc pas sur le séjour. Enfin, une porte d'entrée est percée dans le plus long mur extérieur du couloir, ou à défaut du séjour.

Essais multiples. Le placement étant aléatoire, jusqu'à cinq essais sont réalisés. Nous retenons le premier plan complet, connexe, sans porte interdite et dont toutes les pièces sont accessibles depuis l'entrée sans traverser de pièce privée, ou, à défaut, le meilleur plan selon ces critères. Il faut donc garder à l'esprit que la connexité et l'absence de portes interdites relèvent de l'algorithme de placement, et non des performances du GAN.

3.6 Split Learning en U

Pour l'entraînement collaboratif, le cGAN est découpé en quatre modules (Fig. 1). Chaque agence k dispose d'un encodeur de condition $E_k : c \mapsto e \in \mathbb{R}^{32}$ et de la partie basse du discriminateur $B_k : (v, c) \mapsto s \in \mathbb{R}^{64}$, tandis que le serveur héberge le générateur $G_s(z, e)$ et la partie haute du discriminateur $T(s)$. Un

pas d'entraînement comporte deux phases. Dans la phase du discriminateur, l'agence envoie $e = E_k(c)$ et reçoit en retour $\hat{v} = G_s(z, e)$; elle calcule ensuite $s_{\text{réel}} = B_k(v, c)$ et $s_{\text{faux}} = B_k(\hat{v}, c)$, qu'elle transmet au serveur. Celui-ci calcule la perte, met à jour T et renvoie les gradients de s , grâce auxquels l'agence met à jour B_k . La phase du générateur suit le même aller-retour : la perte adversariale est calculée sur le serveur et la perte de contraintes chez l'agence, qui est la seule à connaître c , puis les gradients redescendent par $\hat{v}$ vers G_s et par e vers E_k . Les programmes v et les conditions c ne quittent donc jamais l'agence.

Nous comparons trois variantes. Dans la variante **SL**, chaque agence conserve ses propres modules E_k et B_k , et les agences sont servies à tour de rôle. Dans la variante **SplitFed**, ces modules sont moyennés (FedAvg) à la fin de chaque époque. La variante **SplitFed+bruit** ajoute en outre un bruit gaussien ($\sigma = 0,5$) à e et s avant leur envoi. Dans notre implémentation, la frontière entre agence et serveur est matérialisée par un détachement explicite des tenseurs et un renvoi manuel des gradients, ce qui permet de mesurer exactement les volumes échangés ; les échanges ont toutefois lieu au sein d'un même processus.

3.7 Données

En l'absence de corpus public de programmes annotés en français, nous avons généré les programmes. Une typologie est d'abord tirée (T1 10%, T2 20%, T3 35%, T4 25%, T5 10%), puis les pièces annexes selon des probabilités simples (balcon 50%, bureau 30%, etc.), et enfin la topologie. Celle-ci suit des préférences qui dépendent du programme : une salle d'eau supplémentaire devient une suite dans 85 % des cas, les WC sont intégrés à la salle d'eau dans 70 % des logements sans couloir contre 15 % des autres, un dressing est attendant à une chambre dans 80 % des cas et une cuisine est fermée dans 35 % des cas lorsqu'il y a un couloir. C'est précisément cette dépendance que le cGAN doit apprendre. Un programme n'est conservé que si le placement parvient à le réaliser entièrement en trois essais au plus, avec un plan connexe et au moins 90 % de portes conformes. Le placement v3 réalise ainsi 98,0 % des programmes tirés, les échecs provenant surtout de rattachements impossibles faute de place autour de la pièce parente. Le jeu final compte 20 000 programmes, dont 984 distincts (104 si l'on ignore la topologie), avec 8,28 pièces en moyenne. Il est réparti à raison de 90 % pour l'apprentissage et 10 % pour le test. Nous dirons qu'un programme

généré est *plausible* lorsqu'il appartient à l'ensemble des programmes réalisables observés.

4 Protocole expérimental

Métriques. Le *respect brut* mesure la part des valeurs imposées que la sortie arrondie du cGAN reproduit exactement, avant le snap. Le *respect dans le plan* applique la même mesure au plan final, en comptant les pièces effectivement placées et en relisant la topologie sur les portes ; le *respect du graphe* la restreint aux variables de topologie, pour les requêtes qui en imposent. La *plausibilité* a été définie plus haut. La *diversité* correspond au nombre de programmes distincts obtenus pour une même requête, et la *diversité du graphe* au nombre de topologies distinctes. La *connectivité* est la part des pièces atteignables depuis le salon en passant par les portes (parcours en largeur). Les *portes légales* désignent la part des portes reliant deux pièces dont l'adjacence est autorisée, et l'*adjacence* la part des pièces ayant au moins un voisin autorisé accessible par une porte. Nous mesurons également la part des plans *sans porte interdite*, l'*accès direct*, c'est-à-dire la part des pièces atteignables depuis l'entrée sans traverser de pièce privée (chambre, salle d'eau, WC, bureau ou dressing), une annexe pouvant toutefois être desservie par sa pièce principale, ainsi que le nombre moyen de *portes par pièce privée*. La *compacité* rapporte la surface des pièces à celle de la boîte englobante, et l'*écart de surface* mesure l'écart relatif à la surface demandée lorsque la requête en précise une. Le *temps* couvre le parsing, la génération et le placement, sans le rendu graphique.

Méthodes de référence. Toutes les méthodes utilisent le même parser, le même placement et le même rendu ; seule change la façon de compléter les éléments libres. Nous comparons la méthode proposée (cGAN+snap) au cGAN sans snap (cGAN brut), à un tirage *aléatoire* uniforme des éléments libres dans $[0, M_i]$, et à un *prior empirique* qui tire un programme du jeu d'apprentissage compatible avec les contraintes, ou le plus proche s'il n'en existe aucun.

Jeux d'évaluation. Au niveau du vecteur, 2 000 conditions sont tirées du jeu de test en masquant chaque composante avec une probabilité de 0,5. Au niveau du plan, nous utilisons 22 requêtes qui couvrent les typologies, les pièces annexes, les négations, trois surfaces imposées, cinq structures imposées (« T4 avec suite parentale », « T3 avec cuisine fermée », « T2 avec WC séparés », etc.) et un cas hors distribution (« appart 5 chambres sans balcon »), chacune générée en 50 variantes.

Table 1. Évaluation au niveau du vecteur (conditions du jeu de test). « – » : respect garanti par construction.

Méthode	Respect brut	Plausibilité	Diversité
Aléatoire	–	42,2 %	74,2 %
Prior empir.	–	99,9 %	40,5 %
cGAN brut	100,0 %	93,6 %	–
cGAN + snap	100,0 %	93,7 %	39,0 %

Split Learning. Le jeu d'apprentissage est réparti entre trois agences selon le nombre de chambres (A : 0 ou 1, B : 2, C : 3 ou 4). Pour rendre la répartition moins tranchée, 70 % des programmes sont attribués à l'agence correspondante et 30 % à une autre. Chaque configuration est entraînée pendant 100 époques et comparée au modèle centralisé au même nombre d'époques. Les modèles sont évalués sur l'ensemble des conditions de test, sur la partie correspondant aux typologies de chaque agence et sur celle des autres agences.

Attaque par inversion. Nous supposons un serveur curieux qui connaît 20 % des couples (programme, s) et entraîne un réseau à reconstruire v à partir de s . Nous mesurons la part des programmes exactement reconstruits sur les 80 % restants, que nous comparons à une référence sans information consistant à prédire le programme le plus fréquent.

Configuration. Les expériences ont été menées avec PyTorch sur Google Colab (GPU), avec l'optimiseur Adam ($\text{lr} = 2 \times 10^{-4}$, $\beta = (0,5; 0,999)$), des lots de 128 et une graine fixée à 42. L'entraînement centralisé dure 8,7 min pour 200 époques.

5 Résultats

5.1 Apprentissage du programme

Le Tableau 1 permet d'isoler la contribution du réseau. Sans snap, le cGAN reproduit 100,0 % des valeurs imposées et produit 93,6 % de programmes plausibles, alors qu'un tirage aléatoire des éléments libres n'en donne que 42,2 %. La plausibilité obtenue vient donc bien de ce que le cGAN a appris de la distribution conditionnelle. Sa diversité (39,0 % de programmes distincts pour une même condition) est par ailleurs comparable à celle d'un tirage dans les données (40,5 %). Le prior empirique reste, lui, plausible par construction, et sur des données synthétiques une simple recherche dans la base constitue une référence difficile à battre. Il suppose cependant de disposer de toute la base en un même lieu, ce qui n'est justement pas possible lorsque les données appartiennent à



Figure 2. Plans générés par cGAN + snap pour quatre requêtes, dont un hors distribution (5 chambres). Murs porteurs en trait épais, portes avec arc de débattement, 1 cellule = 30 cm.

plusieurs agences, alors que le cGAN peut être appris sans les centraliser (§5.5).

5.2 Qualité des plans

Le Tableau 2 et la Fig. 2 portent sur la chaîne complète. La connectivité est très élevée pour toutes les méthodes, mais elle découle du placement et ne peut pas être attribuée au cGAN. Le snap fait passer le respect de la requête dans le plan de 98,4 % à 100,0 %, avec en contrepartie une plausibilité plus faible (80,5 % contre 84,7 %). Cet écart était attendu, puisque le snap impose parfois des combinaisons absentes de la base ; c'est le cas de la requête « appart 5 chambres sans balcon », dont la plausibilité est nulle, le jeu de données s'arrêtant à quatre chambres. Par rapport au tirage aléatoire (17,0 %), le cGAN multiplie la plausibilité par près de cinq, tout en restant en deçà du prior empirique (95,5 %), pour la raison indiquée plus haut.

Notons qu'une première série d'expériences avait révélé une incohérence dans nos règles. Un WC ne pouvait pas s'ouvrir sur le séjour, si bien que le filtre de réalisabilité écartait tous les T2 sans couloir dont les WC étaient séparés de la salle d'eau, et que la requête « T2 avec WC séparés » obtenait une plausibilité nulle. Après correction, cette liaison étant désormais autorisée en l'absence de couloir, la part de programmes réalisables est passée de 88,8 % à 98,0 %

Table 2. Évaluation des plans complets : 22 requêtes × 50 variantes, placement v3. Moyennes ; l'intervalle de confiance à 95 % est donné en points de pourcentage pour la méthode proposée.

Méthode	Respect (plan)	Plausibilité	Diversité	Connectivité	Portes légales	Adjacence
Aléatoire	99,8 %	17,0 %	89,5 %	100,0 %	98,5 %	93,6 %
Prior empir.	100,0 %	95,5 %	47,2 %	100,0 %	100,0 %	89,4 %
cGAN brut	98,4 %	84,7 %	52,3 %	100,0 %	99,1 %	89,0 %
cGAN + snap	100,0 % ±0,0	80,5 % ±2,3	51,9 % ±1,0	100,0 % ±0,0	99,1 % ±0,2	89,1 % ±1,0

Table 3. Qualité architecturale selon le placement (mêmes programmes, générés par cGAN + snap).

	v2	v3, 1 essai	v3
Portes légales	98,7 %	99,1 %	99,1 %
Sans porte interdite	96,5 %	100,0 %	100,0 %
Accès direct	98,6 %	99,3 %	100,0 %
Portes / pièce privée	1,70	1,24	1,22
Compacité	63,7 %	73,6 %	73,9 %
Porte d'entrée	0,0 %	99,9 %	99,9 %
Écart de surface	20,5 %	4,7 %	4,6 %
Temps (ms)	38,3	1,8	2,2

Table 4. Graphe prédit par le cGAN ou déduit par des règles fixes (mêmes pièces, placement v3). Le respect du graphe porte sur les cinq requêtes qui imposent une structure.

	Règles fixes	cGAN
Respect du graphe imposé	50,8 %	99,4 %
Respect du prompt (plan)	97,6 %	100,0 %
Diversité du graphe	14,0 %	32,1 %
Plausibilité	82,4 %	80,5 %
Accès direct	100,0 %	100,0 %
Portes / pièce privée	1,25	1,22

Table 5. Effet de λ (Éq. 2), 60 époques.

λ	Respect brut	Plausibilité	Diversité
0	94,7 %	92,4 %	38,8 %
1	97,0 %	90,8 %	35,3 %
5	99,4 %	95,1 %	34,2 %
15	99,8 %	94,8 %	36,6 %
50	100,0 %	94,3 %	34,1 %

et la plausibilité de cette requête à 0,84. Les résultats présentés ici sont ceux du système corrigé.

Le Tableau 3 permet d'évaluer l'effet du placement à programmes identiques. Le placement v3 réduit le nombre moyen de portes des pièces privées de 1,70 à 1,22, l'excédent par rapport à une porte correspondant aux suites et aux salles d'eau qui intègrent les WC. Il améliore la compacité, qui passe de 63,7 % à 73,9 %, ajoute une porte d'entrée et ramène l'écart à la surface demandée de 20,5 % à 4,6 %, tout en étant plus de dix fois plus rapide. Les essais multiples éliminent quant à eux les derniers cas de pièces traversées, l'accès direct passant de 99,3 % avec un seul essai à 100,0 % avec cinq.

Temps de génération. Un plan est produit en 2,4 ms en moyenne (médiane de 2,1 ms, minimum de 0,7 ms et maximum de 12,8 ms), dont moins de 0,1 ms pour le cGAN lorsque l'inférence est faite par lots. Le rendu en PNG ajoute 212,8 ms, de sorte que c'est lui, et non le réseau ni le placement, qui domine le temps total. À titre de comparaison, le prototype du mémoire annonçait 0,376 s par plan, rendu compris.

5.3 Prédiction du graphe de bulles

Le Tableau 4 compare, à pièces égales, la topologie prédite par le cGAN à celle que produisent des règles fixes. Ces règles ne peuvent satisfaire une demande de structure que si elle correspond à leur choix par défaut, alors que le graphe prédit, une fois les contraintes

rétablies, la satisfait dès lors que le placement parvient à le réaliser (Fig. 3). Sur les cinq requêtes structurales, le graphe demandé est obtenu dans 99,4 % des cas avec la prédiction, contre 50,8 % avec les règles fixes. Ces dernières échouent par exemple systématiquement sur « T3 avec cuisine fermée », et dans 92 % des cas sur « T2 avec WC séparés ». La prédiction fait en outre plus que doubler la diversité des graphes obtenus pour une même requête (32,1 % contre 14,0 %), au prix d'un léger recul de la plausibilité (80,5 % contre 82,4 %) qui s'explique par un espace de programmes plus vaste. Les garanties apportées par le placement restent inchangées. À titre de comparaison, le placement v2, qui ne tient pas compte du graphe, n'en réalise que 52,0 %.

5.4 Influence du poids λ

Le conditionnement suffit à lui seul ($\lambda = 0$) à reproduire 94,7 % des valeurs imposées. La perte de contraintes comble l'essentiel de l'écart restant, puisque le respect dépasse 99 % dès $\lambda = 5$. Elle ne

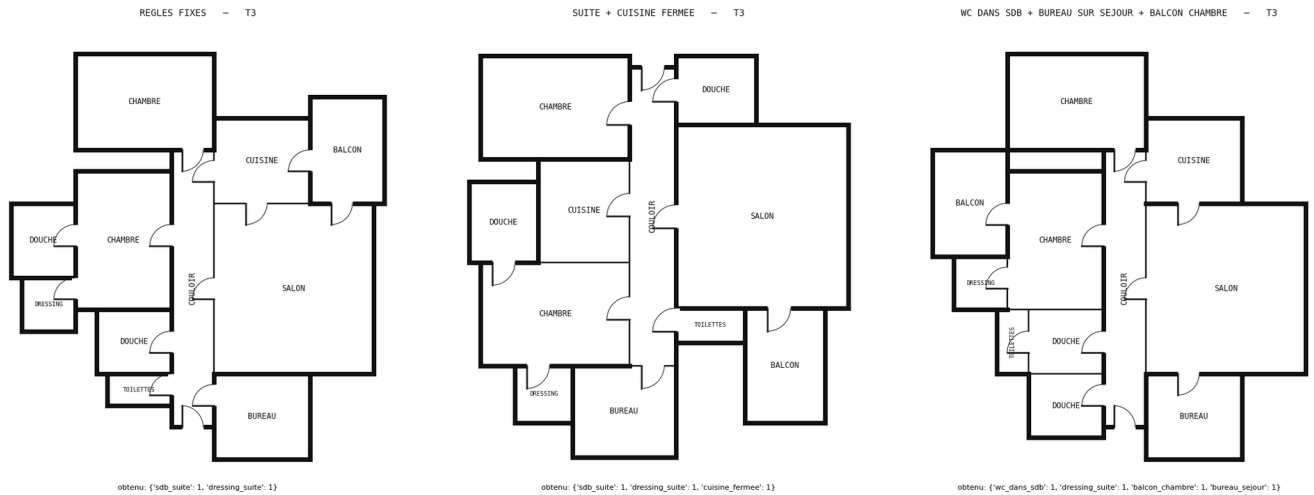


Figure 3. Un même programme (T3 avec deux salles d'eau, dressing, balcon et bureau) réalisé avec trois topologies : règles fixes ; suite parentale et cuisine fermée ; WC dans la salle d'eau, bureau ouvert sur le séjour et balcon desservi par une chambre. Sous chaque plan figure la topologie relue sur les portes.

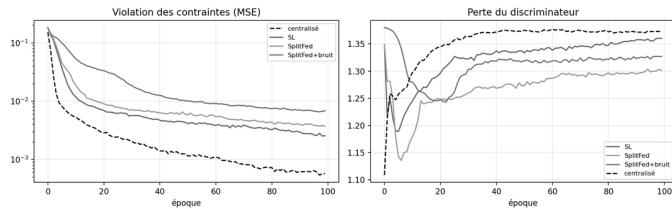


Figure 4. Convergence des variantes de Split Learning comparée à l'entraînement centralisé.

dégrade pas la plausibilité (92,4 % pour $\lambda = 0$ et 94,8 % pour $\lambda = 15$), mais s'accompagne d'une légère baisse de la diversité, qui passe de 38,8 % à 34,1 % pour $\lambda = 50$, le générateur devenant un peu plus prudent. La valeur retenue, $\lambda = 15$, se situe sur le palier où le respect est maximal.

5.5 Entraînement en Split Learning

Le Tableau 6 répond à deux questions. Il s'agit d'abord de savoir si la collaboration apporte quelque chose à une agence par rapport à un entraînement local, en particulier sur les typologies qu'elle connaît peu, puis de mesurer l'écart qui subsiste avec un modèle centralisé, lequel suppose de partager les données.

Une agence qui travaille seule obtient un modèle satisfaisant sur ses propres typologies, mais moins fiable sur les autres, avec un respect des contraintes qui descend à 93,6 % sur ces dernières. SplitFed corrige en grande partie ce défaut, avec 96,4 % de respect et 98,8 % de plausibilité sur les typologies des autres agences, et atteint globalement 97,9 %, contre 100,0 % pour le modèle centralisé. L'écart résiduel apparaît aussi sur les courbes de convergence (Fig. 4), où la violation des contraintes reste environ six fois plus élevée

qu'en centralisé après 100 époques. La plausibilité est en revanche du même ordre, voire légèrement supérieure, ce qui laisse penser que le générateur est un peu plus conservateur. La variante SL, dans laquelle chaque agence garde son propre encodeur de condition, donne les moins bons résultats (93,4 % de respect et 84,7 % de plausibilité), car le générateur partagé reçoit des représentations de la condition qui diffèrent d'une agence à l'autre. Dans ce contexte de données hétérogènes, la mise en commun des modules client opérée par SplitFed s'avère donc indispensable.

Coût. Les modules situés chez les agences représentent 31,9 % des paramètres (18 208 contre 38 800 sur le serveur). SplitFed échange 37,36 Mo d'activations et de gradients par époque, soit 2100 octets par exemple, auxquels s'ajoutent 0,44 Mo de synchronisation. Un FedAvg échangeant le modèle complet coûterait 1,37 Mo par époque, soit environ vingt-sept fois moins, et le jeu d'apprentissage brut ne pèse lui-même qu'environ 1 Mo. Avec un modèle aussi petit et des données aussi compactes, le Split Learning ne se justifie donc pas par un gain de bande passante. Son intérêt tient à l'allègement du calcul chez les agences et, sous certaines conditions, à la confidentialité. La situation serait différente avec des modèles de grande taille [15].

Fuite d'information. Lorsque 20 % des couples sont connus, l'attaque par inversion reconstruit exactement 52,9 % des programmes à partir des activations de SplitFed, contre 3,1 % pour la référence sans information, et seulement 0,4 % lorsque le bruit est activé. Sans protection, les activations échangées protègent donc mal les programmes : l'attaquant

Table 6. Entraînement collaboratif sur trois agences non-IID (100 époques). Respect brut des contraintes / plausibilité, sur l'ensemble des conditions de test, sur le segment propre de chaque agence et sur les segments des autres agences.

Configuration	Global		Segment propre		Segments étrangers	
	Respect	Plaus.	Respect	Plaus.	Respect	Plaus.
Centralisé (référence, non confidentiel)	100,0 %	94,5 %	100,0 %	95,3 %	100,0 %	95,4 %
Local seul (sans collaboration)	97,0 %	95,6 %	97,5 %	98,8 %	93,6 %	94,1 %
SL (modules client privés)	93,4 %	84,7 %	89,3 %	96,2 %	90,2 %	89,9 %
SplitFed	97,9 %	95,6 %	96,5 %	98,8 %	96,4 %	98,8 %
SplitFed + bruit ($\sigma = 0,5$)	95,8 %	98,2 %	93,1 %	98,8 %	93,1 %	98,8 %

en retrouve plus d'un sur deux à l'identique et retrouve 95,0 % des composantes, contre 69,5 % pour la référence. Le bruit ramène l'attaque au niveau de cette référence (68,0 % des composantes), ce qui signifie que l'inverseur n'extrait plus d'information exploitable, pour un coût modéré sur la qualité (respect global de 95,8 % contre 97,9 %). Ces observations rejoignent les attaques décrites dans la littérature [16, 17] : le Split Learning n'assure la confidentialité que s'il est accompagné d'une protection explicite. Notons aussi que notre bruit gaussien n'offre pas de garantie formelle, contrairement à la confidentialité différentielle.

6 Discussion

Répartition des rôles entre le réseau et l'algorithme.

Notre approche confie au cGAN ce qui relève d'une distribution, par exemple les pièces qui accompagnent habituellement un T3 avec bureau, et à un algorithme ce qui relève de contraintes strictes, comme l'accessibilité de toutes les pièces. Ce partage rend les garanties vérifiables, mais il implique aussi que la qualité géométrique du plan dépend entièrement du placement. Or, même en v3, l'emprise reste irrégulière (compacité de l'ordre de 0,75), l'orientation et l'éclairage naturel ne sont pas pris en compte, et le graphe se limite à un arbre de distribution à six degrés de liberté. Cette représentation est bien plus restreinte que les graphes quelconques de House-GAN [6], mais elle garantit que tout graphe généré est valide et réalisable, et elle peut être pilotée par une requête en langage courant. Le placement peut néanmoins ne pas réussir à réaliser un rattachement demandé lorsque la pièce parente n'a plus de place, ce que mesure justement le respect du graphe.

Retour sur le prototype v1. Le prototype du mémoire affichait une connectivité de 1,00 et une conformité d'adjacence de 0,92. Avec le recul, ces

chiffres provenaient pour l'essentiel de la fonction de placement, puisque le gradient des pertes d'adjacence et SSIM ne pouvait pas la traverser pour atteindre le générateur. La version actuelle rend ce point explicite et mesure à part la contribution du réseau (Tableau 1).

Limites. Ce travail présente plusieurs limites. Les programmes d'entraînement sont synthétiques, de sorte que le modèle apprend la distribution que nous avons définie et non celle de plans réels; l'étape suivante consistera à extraire des programmes de corpus de plans réels, comme RPLAN [11], ou de plans fournis par des agences partenaires. Les règles d'adjacence, les liaisons interdites et les plages de dimensions ont été fixées à la main à partir des usages courants du logement, sans s'appuyer sur une norme. Le Split Learning est simulé au sein d'un seul processus, si bien que la latence du réseau et le chiffrement des échanges (TLS) ne sont pas mesurés. L'attaque étudiée reste simple, et des attaques plus fortes comme PCAT [17] ou des défenses dédiées comme NoPeek [19] restent à évaluer. Enfin, la diversité et la plausibilité ne remplacent pas l'avis d'architectes, qu'il faudrait recueillir lors d'une évaluation qualitative.

7 Conclusion

Nous avons présenté un générateur de plans 2D piloté par une requête textuelle, qui associe un GAN conditionnel chargé de compléter le programme à un placement procédural garantissant la validité fonctionnelle du plan, et nous l'avons entraîné en Split Learning en U entre trois agences sans que leurs programmes ne quittent leurs locaux. Les plans obtenus respectent 100,0 % des contraintes de la requête, ne comportent aucune porte interdite et sont produits en quelques millisecondes. La prédiction du graphe de bulles porte le respect des structures demandées à 99,4 %. Entraîné en SplitFed, le modèle s'approche du modèle centralisé (97,9 % contre

100,0%) et dépasse les modèles locaux, à condition toutefois de brouter les activations échangées, faute de quoi plus de la moitié des programmes peuvent être reconstruits.

Plusieurs prolongements sont envisagés : l'apprentissage sur des programmes issus de plans réels, l'extension du conditionnement à l'emprise et à l'orientation, la mise en place de protections offrant des garanties formelles (confidentialité différentielle, NoPeek [19]), un déploiement du protocole sur un véritable réseau et l'export des plans vers les logiciels de CAO aux formats DXF ou IFC.

Disponibilité du code

Le code source, le notebook ArchiGAN_SL_v2.ipynb et les poids du modèle entraîné sont disponibles à l'adresse https://github.com/Kenny-19/Archi_GAN. Le notebook permet de reproduire l'ensemble des expériences ainsi que les tableaux et figures de cet article (graine fixée à 42). Une démonstration en ligne, qui exécute le générateur directement dans le navigateur, est accessible à l'adresse <https://archigansite.vercel.app>.

Remerciements

L'auteur remercie le Professeur Antoine Bagula (directeur) et l'Ass. Benjamin Oleko (encadreur) pour leurs orientations scientifiques, ainsi que l'Université Nouveaux Horizons.

Conflits d'intérêt

L'auteur déclare n'avoir aucun conflit d'intérêt.

Références

- [1] P. Gire. Conception assistée par ordinateur dans le domaine mécanique. *Revue française de Gestion industrielle*, 24 :1–14, 1983.
- [2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11) :139–144, 11 2020.
- [3] Weixin Huang and Hao Zheng. Architectural drawings recognition and generation through machine learning. In *Proceedings of the 2018 ACADIA Conference*. ACADIA, 2018.
- [4] Hao Zheng, Keyao An, Jingxuan Wei, and Yue Ren. Apartment floor plans generation via generative adversarial networks. In *Proceedings of the 25th International Conference of the Association for Computer-Aided Architectural Design Research in Asia (CAADRIA)* 2020, pages 599–608. Association for Computer-Aided Architectural Design Research in Asia (CAADRIA), 2020.
- [5] Stanislas Chaillou. ArchiGAN : Artificial intelligence x architecture. In *Architectural Intelligence*, pages 117–127. Springer, 2020.
- [6] Nelson Nauata, Kai-Hung Chang, Chin-Yi Cheng, Greg Mori, and Yasutaka Furukawa. House-GAN : Relational generative adversarial networks for graph-constrained house layout generation. In *European Conference on Computer Vision (ECCV)*, pages 162–177, 2020.
- [7] Otkrist Gupta and Ramesh Raskar. Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications*, 116 :1–8, 2018.
- [8] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health : Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv :1812.00564*, 2018.
- [9] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. SplitFed : When federated learning meets split learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8485–8493, 2022.
- [10] Ruizhen Hu, Zeyu Huang, Yuhang Tang, Oliver van Kaick, Hao Zhang, and Hui Huang. Graph2Plan : Learning floorplan generation from layout graphs. *ACM Transactions on Graphics*, 39(4) :118 :1–118 :14, 2020.
- [11] Wenming Wu, Xiao-Ming Fu, Rui Tang, Yuhang Wang, Yu-Hao Qi, and Ligang Liu. Data-driven interior plan generation for residential buildings. *ACM Transactions on Graphics*, 38(6) :234 :1–234 :12, 2019.
- [12] Mathias Bank, Viktoria Sandor, Kristina Schinegger, and Stefan Rutzinger. Learning spatiality : A GAN method for designing architectural models through labelled sections. In *Proceedings of eCAADe 40*, pages 611–619. University of Innsbruck, 2022.
- [13] Dongyun Kim, George Guida, and Jose Luis García Del Castillo y López. PlacemakingAI : Participatory urban design with generative adversarial networks. In *Proceedings of the 27th International Conference of the Association for Computer-Aided Architectural Design Research in Asia (CAADRIA)*, volume 2, pages 485–494. Association for Computer-Aided Architectural Design Research in Asia (CAADRIA), 2022.
- [14] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.
- [15] Abhishek Singh, Praneeth Vepakomma, Otkrist Gupta, and Ramesh Raskar. Detailed comparison of communication efficiency of split learning and

- federated learning. *arXiv preprint arXiv :1909.09145*, 2019.
- [16] Dario Pasquini. Unleashing the tiger : Inference attacks on split learning. *arXiv preprint arXiv :2012.02670v5*, 2022.
- [17] Xinben Gao and Lan Zhang. PCAT : Functionality and data stealing from Split Learning by pseudo-client attack. In *USENIX Security Symposium*, 2023.
- [18] Ege Erdoğan, Alptekin Küpçü, and A. Ercüment Çiçek. SplitGuard : Detecting and mitigating training-hijacking attacks in split learning. In *Proceedings of the 21st Workshop on Privacy in the Electronic Society (WPES)*, pages 1–13, New York, NY, USA, 11 2022. ACM.
- [19] Praneeth Vepakomma, Abhishek Singh, Otkrist Gupta, and Ramesh Raskar. NoPeek : Information leakage reduction to share activations in distributed deep learning. In *IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 933–942, 2020.
- [20] Zhanyi Hu, Tianchen Zhou, Bingzhe Wu, Cen Chen, and Yanhao Wang. A review and experimental evaluation on split learning. *Future Internet*, 17(2) :87, 2025.
- [21] Eric Samikwa, Antonio Di Maio, and Torsten Braun. ARES : Adaptive resource-aware Split Learning for internet of things. *Computer Networks*, 218 :109380, 2022.